Laboratoire 3
Acquisition, événements et télémétrie

Eren Arikan · 300357356
Antony Zuze · 300221295
Section A02 · Automne 2026

Rapport de réalisation et de validation logicielle
Énoncé v1.3 du 1er novembre 2026

1. Objectif et portée de la validation

Le laboratoire remplace la boucle et la machine à états assembleur par une application C, tout en conservant les pilotes des laboratoires 1 et 2. Le potentiomètre fournit la consigne ; un ADC à deux voies et un DMA circulaire acquièrent la consigne et la température. Une file d’événements relie acquisition, commande et télémétrie. Une routine Thumb convertit les fronts de l’encodeur en vitesse Q16.16.

Élément	Résultat obtenu
Projet STM32CubeIDE	Construction Debug : 0 erreur, 0 avertissement. Firmware lié à -O0 et -O2.
File et calculs	12/12 essais de file sous Linux et QEMU ; 5/5 cas de vitesse assembleur.
Commande et émission	Tests hôte des transitions, rampe, urgence injectée, filtrage, trames et saturation réussis.
Comparaison assembleur/C	8/8 cas PWM à -O0 et -O2 sous QEMU, TIM1 simulé en RAM.
Validation matérielle	ST-LINK détecté le 10 novembre ; cible à 0,00 V, connexion SWD impossible. Essais matériels, Windows et démonstration à effectuer.

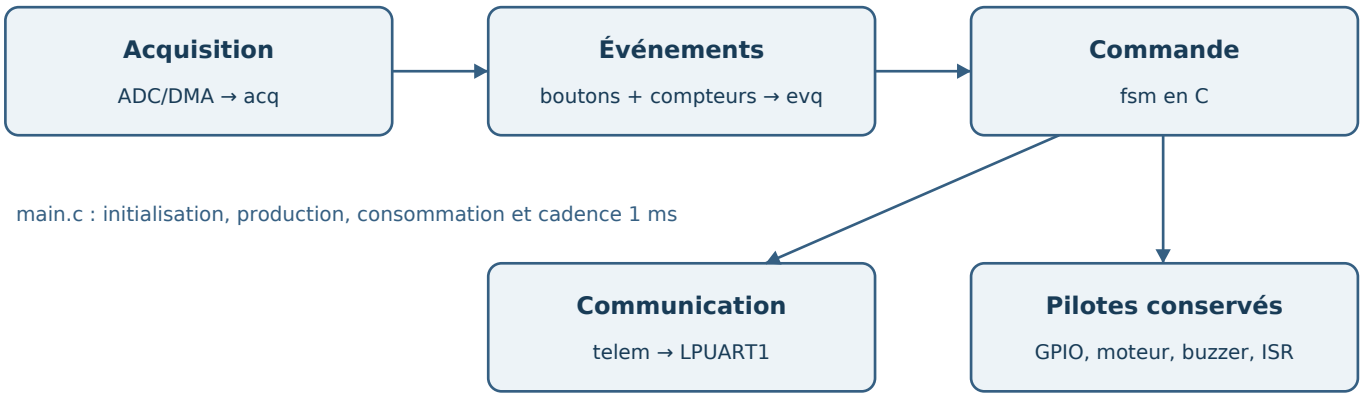
Lecture des preuves. « Réussi sous Linux » désigne un essai du code sur l’hôte. « Réussi sous QEMU » désigne une exécution du code ARM sur un processeur émulé. Les captures sont des vues des journaux enregistrés. Les données de télémétrie synthétiques ne sont pas présentées comme une acquisition sur carte.

Hypothèses retenues : MSI/HCLK/PCLK1 = 4 MHz ; fenêtre encodeur = 250 ms ; 748 fronts par tour ; ADC 12 bits et VREF = 3 300 mV ; deux voies ; 16 échantillons pour le potentiomètre et 64 pour la température en version 2. Le point thermique 1 831 mV à 25,0 °C est une référence initiale du complément, à étalonner sur la plateforme.

2. Conception et interfaces · L3-B

Le problème consiste à maintenir la réactivité des commandes et de l’arrêt d’urgence, sans attente de transmission série, avec une mémoire fixe et des modules testables sans matériel. Une tâche lente ou une file pleine ne doit pas empêcher la mise en sécurité. Les accès aux registres des anciens pilotes restent exclusivement en assembleur.

Critère	Scrutation directe	File d’événements retenue
Couplage	Chaque lecture peut déclencher directement les sorties.	Producteurs et consommateur communiquent par type, valeur et date.
Testabilité	Le contrôle dépend souvent des broches et du temps réel.	evq et acq fonctionnent en mémoire ; fsm accepte des événements injectés.
Mémoire/temps	Peu de stockage ; ordre implicite de traitement.	32 entrées fixes, opérations push/pop en temps constant.
Surcharge	Un événement peut être écrasé implicitement.	Refus explicite, compteur de débordements et contenu FIFO conservé.



Les registres nouveaux restent dans adc_dma.c et telem.c ; evq/acq sont testables sur hôte.

Contrats des modules

evq stocke 32 structures de 12 octets, soit 384 octets de données. Tous ses appels proviennent de main ; les ISR ne manipulent pas la file. **acq** reçoit une source volatile, un pas, un nombre d’échantillons et une calibration. Son en-tête ne révèle pas ses champs. **adc_dma** connaît les registres et publie la position de la prochaine paire complète. **fsm** consomme un événement à la fois et regroupe les appels aux sorties dans fsm_sorties. **telem** reçoit un état exposé, construit une ligne CSV et la met en attente sans attendre la liaison.

Le choix de la file rend l’ordre des événements visible et permet les essais de saturation. Son coût est borné : 384 octets pour les entrées, plus cinq compteurs de 32 bits. Elle ne remplace pas l’arrêt matériel : l’ISR E-Stop coupe déjà le moteur. Une voie prioritaire transmet également EV_ESTOP à la machine à états lorsque la file est pleine.

3. Migration en C et commande

Conservation des pilotes et AAPCS

Les fichiers gpio.s, buttons.s, estop.s, motor.s, encoder.s, systick.s et buzzer.s ont été repris du projet Lab 2 local retrouvé. Une comparaison octet par octet confirme leur conservation, ainsi que celle des deux fichiers de constantes assembleur. main.s et fsm.s sont remplacés par main.c et fsm.c. Le démarrage, le script de liens et les fichiers de support générés sont conservés. Les empreintes sont consignées dans preservation_lab2.txt.

Les prototypes de lab1.h et lab2.h conservent les paramètres 32 bits et les pointeurs attendus. estop_flag, enc_count, sens_mesure, ticks_ms, tick_1ms et buzzer_continu restent volatile. Le programme appelle d'abord les initialisations GPIO, buzzer, moteur, encodeur, E-Stop et SysTick, puis la couche C. fsm_init impose l'arrêt et la DEL rouge ; telem_entete prépare la ligne H.

État / événement	Transition et sorties
ARRÊT + User	Départ en avant, puis en arrière au départ suivant. PWM repart de 0.
MARCHE + User	Passage par ARRÊT ; PWM, AI1/AI2 et STB coupés avant toute nouvelle direction.
MARCHE + consigne	Consigne limitée à 0..100 %. Rampe de 1 point toutes les 10 ms.
Tout état + E-Stop	ARRÊT_URGENCE, PWM/STB coupés, rouge clignotante, son continu.
Urgence + Touch En	Acquittement seulement si E-Stop est relâché ; retour en ARRÊT, jamais en marche.
Touch En hors urgence	Bascule de touch_enabled et un bip, sans modification de la consigne.

Priorité de l'urgence et limites de concurrence

Le drapeau E-Stop reste verrouillé jusqu'à l'acquittement. main notifie chaque verrouillage une fois et traite l'urgence avant les appuis ordinaires. Si push refuse l'événement, le débordement est compté et l'urgence est traitée directement. fsm_sorties vérifie aussi le drapeau avant et après les appels de sortie afin de recouper les sorties si l'ISR a préempté une écriture. Un essai hôte injecte précisément cette situation pendant motor_set_pwm.

Les lectures alignées de mots de 32 bits sont atomiques individuellement sur cette cible ; volatile impose les accès, sans rendre une séquence lecture-modification-écriture atomique. Le bit tick_1ms peut fusionner plusieurs ticks pendant une opération de diagnostic. Le fonctionnement à la cadence demandée et la latence E-Stop inférieure à 1 ms devront donc être mesurés sur la carte, notamment pendant la rampe et l'émission série.

4. Acquisition ADC/DMA et télémétrie

Configuration de l’acquisition

PC0 et PC1 sont placées en mode analogique sans résistances de rappel. ADC1 utilise HCLK/1 ; le programme quitte DEEPPWD, active le régulateur, attend via SysTick, réalise l’étalonnage simple entrée et attend ADRDY. Les conversions continues alternent IN1 puis IN2, avec 640,5 cycles d’échantillonnage et 12,5 cycles de conversion. DMA1 canal 1, requête DMAMUX 5, transfère des demi-mots dans le tampon circulaire.

Paramètre	Valeur / calcul
ADC CFGR, SMPR1, SQR1	0x3003 ; 0x1F8 ; 0x2041 (champs programmés à l’initialisation).
DMA CCR et CNDTR	0x5A1 ; 128 demi-mots = 64 paires, soit 256 octets.
Débit par voie	4 000 000 / [2 × (640,5 + 12,5)] = 3 062,79 échantillons/s.
Durée de l’historique	16 échantillons : 5,224 ms ; 64 : 20,896 ms.
Vérification prévue	Différence de adc_dma_tours sur 1 s × 64 ; vérifier aussi CNDTR et le tampon.

Le compteur de tours observe le drapeau TCIF1 à chaque tick. Il n’est exact que si la boucle observe chaque fin de tour : plusieurs tours entre deux lectures ne sont pas récupérables à partir d’un seul bit. Aucun débit réel n’a été mesuré. Le curseur exclut une paire dont IN2 n’est pas encore transférée ; l’acquisition attend un premier tour avant de publier la consigne.

LPUART1 et format

PWR_CR2.IOSV alimente PG7/PG8, configurées en AF8. LPUART1 utilise PCLK1 à 4 MHz, 115 200 bauds, 8N1, sans contrôle de flux. BRR = round(256 × 4 000 000 / 115 200) = 8 889 = 0x22B9. La FIFO matérielle est activée. La file logicielle contient 512 octets ; une ligne est réservée entièrement avant copie. Une file pleine rejette donc une ligne complète au lieu de produire une trame tronquée.

```
H,ticks_ms,etat,sens,consigne_pct,pwm_pct,temp_c_x10,
vitesse_rpm_x100,nb_evenements,evq_debordements
```

Chaque ligne T porte les mêmes neuf champs, puis CRLF. À 5 Hz, une ligne d’environ 60 caractères représente 300 octets/s, soit environ 2,6 % de la liaison 8N1. telem_step écrit au plus huit caractères prêts et revient immédiatement ; elle est appelée aussi entre les ticks. Le formateur utilise uniquement des entiers. La conversion de vitesse d’affichage emploie un produit 64 bits pour éviter le débordement avant le décalage.

Le vérificateur Python fourni a été corrigé pour faire échouer réellement une cadence incorrecte, un en-tête absent, moins de trois trames ou une terminaison incorrecte. Il gère aussi le débordement 32 bits de ticks_ms. Les 150 trames de test respectent le format et les dates synthétiques à 200 ms ; elles ne valident pas le débit réel du port série.

5. Objet d’acquisition et évolution · L3-C

acq_t est un type incomplet dans acq.h. Ses champs se trouvent uniquement dans acq.c ; acq_instance fournit deux objets statiques, sans allocation dynamique. Chaque instance possède sa source, son pas, sa longueur de moyenne, son gain, son décalage et, en V2, son état de filtrage et son curseur. Les appels de lecture restent acq_brut, acq_mv et acq_valeur.

	Version 1	Version 2
Moyenne	16 échantillons par voie sur le tampon complet.	Potentiomètre : 16 derniers ; température : 64 derniers, avec curseur circulaire.
Conversion	Relation linéaire indépendante par instance.	Même relation ; zone morte et hystérésis configurables.
Changement appelant	Initialisation de deux instances.	Seule l’initialisation et la capacité du tampon changent ; lectures/fsm identiques.
État privé ARM	24 octets par instance.	56 octets par instance, soit 112 pour les deux.
Trace de version	Tag local acq-v1 ; baseline explicite.	Tag local acq-v2 ; tests et comparaison fournis.

La moyenne est arrondie à l’entier le plus proche. La tension est calculée par (brut × 3 300 + 2 047) / 4 095. Le filtre applique une zone morte de 3 % aux deux extrémités avant l’arrondi final ; la sortie n’est mise à jour que lorsque l’écart dépasse deux points. Les cas bruts 0, 122, 123, 3 972, 3 973 et 4 095 donnent respectivement 0, 0, 3, 97, 100 et 100 % lorsque le filtre est réinitialisé.

Les essais vérifient le passage d’une moyenne de 16 à 64, l’enroulement, le maintien à 50 % pour une entrée de 52 %, puis le passage à 53 %. Réinitialiser le gain du potentiomètre de 100/3 300 à 50/3 300 modifie sa sortie de 50 à 25 sans changer celle de la température. L’annexe impérative compilable emploie des variables globales dédiées ; ajouter une voie y oblige à multiplier état et fonctions.

Température et conditionnement externe

La calibration initiale est t10 ≈ 250 - (mv - 1 831) × 10 000 / 10 900. Elle doit être ajustée avec un thermomètre. Un pas ADC vaut 0,806 mV, soit environ 0,0739 °C avec la pente de -10,9 mV/°C. Sur 15..40 °C, le signal varie d’environ 272,5 mV. Un gain externe de 4 donnerait 1,09 V de variation et 0,0185 °C par pas, mais exige un décalage : amplifier directement 1,831 V dépasserait 3,3 V.

Exemple calculé : Vout = 1,65 V - 4 × 10,9 mV/°C × (T - 25 °C), soit 2,086 V à 15 °C et 0,996 V à 40 °C. Le gain n’améliore pas le bruit propre du capteur et ajoute celui du circuit. Une moyenne de 64 au lieu de 16 divise théoriquement l’écart-type par deux si le bruit est indépendant, avec une fenêtre quatre fois plus longue.

Limites : capacité statique de deux instances, pas de polymorphisme, source valide et paramètres numériques bornés requis. Les lectures demi-mot sont indivisibles, mais un tour DMA entier n’est pas un instantané atomique. Le filtre peut donc réunir des valeurs prises à des instants voisins pendant la lecture.

6. Vitesse en virgule fixe et AAPCS

Pour une fenêtre de 250 ms, la vitesse de sortie est fronts × 240 / 748 tr/min. Le format Q16.16 représente cette vitesse par un entier non signé divisé par 65 536. La constante idéale est 21 027,5935829 ; l'arrondi retenu est **k = 21 028**. L'erreur relative est +0,00193278 %, inférieure à 0,05 %. Pour zéro front, le résultat est exactement zéro.

Fronts	Référence tr/min	Q16.16 obtenu	Tr/min obtenu
0	0.000000	0	0.000000
1	0.320856	21028	0.320862
748	240.000000	15728944	240.004639
1870	600.000000	39322360	600.011597
65535	21027.272727	1378069980	21027.679138

Les cinq résultats ont été obtenus en exécutant vitesse.s sous QEMU Cortex-M33 à -O0 et -O2. Le produit maximal demandé, 65 535 × 21 028 = 1 378 069 980, tient sur 32 bits. La limite exacte avant débordement est floor((2³² - 1) / 21 028) = 204 249 fronts. La routine n'effectue pas de saturation au-delà de ce contrat.

```
movw r1, #21028
mul r0, r0, r1
bx lr
```

Il s'agit d'une fonction feuille de 10 octets d'instructions : R0 reçoit le nombre de fronts et retourne le résultat ; seul R1 est utilisé comme temporaire. Aucun registre préservé R4-R11 n'est modifié et SP ne change pas.



Capture P3 : R0 = 1 870 à l'entrée ; R0 = 39 322 360 au retour. SP = 0x30003F98 et SP & 7 = 0. enc_count est le symbole réel du pilote conservé ; sa valeur 1 870 a été injectée par le banc. Vue intégrale dans les annexes et journal GDB brut fourni.

7. Comparaison assembleur / C · L3-A

La routine conservée choisie est motor_set_pwm. Elle sature un uint32_t à 100, calcule CCR1 = pct × 200 / 100, écrit TIM1_CCR1 et retourne la valeur écrite. Sa version C possède le même prototype. La production continue d'appeler la version assembleur ; la version C sert uniquement au banc.

Entrée pct	Retour ASM	Retour C	CCR1 ASM / C
0	0	0	0 / 0
1	2	2	2 / 2
25	50	50	50 / 50
50	100	100	100 / 100
75	150	150	150 / 150
99	198	198	198 / 198
100	200	200	200 / 200
4 294 967 295	200	200	200 / 200

Résultat : zéro écart pour les huit cas aux deux niveaux d'optimisation. Dans ce banc QEMU, seul le symbole d'adresse TIM1 est redirigé vers une zone RAM réservée ; le code de calcul et les écritures sont exécutés. Ces résultats vérifient le comportement logiciel et les effets de bord sur une mémoire simulée, pas la PWM physique.

Symbole / mesure	-O0	-O2
motor_set_pwm (assembleur)	24 octets	24 octets
motor_set_pwm_c	64 octets	20 octets
fsm_etat	68 octets	56 octets
fsm_traiter	364 octets	372 octets
Cycles DWT ASM / C / fsm	Non mesurés sur STM32	Non mesurés sur STM32

Les tailles sont celles des symboles ELF : elles peuvent inclure un littéral interne, tandis qu'un pool externe au symbole n'y est pas compté. À -O2, le compilateur sait que pct ≤ 100 ; il remplace multiplication et division par un décalage d'un bit et supprime les variables locales sur la pile. À -O0, il conserve une trame et davantage d'accès mémoire. L'écriture volatile de CCR1 reste présente dans les deux cas.

Le banc sur carte est prêt : commande b en ARRÊT, STB maintenu bas, lecture de DWT_CYCCNT avant/après et soustraction d'une mesure vide. Il expose les cycles moyens ASM/C et ceux de fsm_etat. Les interruptions restent actives ; plusieurs répétitions sont nécessaires pour identifier leur effet. Aucune durée QEMU n'est convertie en cycles STM32.

8. Outils, encodages et optimisation · L3-D

Le projet a été compilé avec GNU Tools for STM32 14.3.1 pour Cortex-M33/Thumb, en gnu11, avec -Wall -Wextra. La construction en ligne de commande ajoute -Werror. Les listes -d -S de motor_set_pwm_c et de fonctions de fsm.c sont fournies pour -O0 et -O2. Les captures GDB montrent paramètres, retour, pile et variable volatile en mémoire sous QEMU.

Source C	Instruction Thumb -O2	Encodage / octets en mémoire
if (pct > 100u)	cmp r0, #100	0x2864 / 64 28
pct * 200u / 100u	lsls r0, r0, #1	0x0040 / 40 00
TIM1->CCR1 = ccr	str r0, [r3, #52]	0x6358 / 58 63

Pour **0x6358**, le format STR immédiat 16 bits est `01100 imm5 Rn Rt`. Ici imm5 = 13, Rn = 3 et Rt = 0. Le déplacement vaut 13 × 4 = 52 octets = 0x34, l'offset de CCR1. R3 porte la base TIM1 = 0x40012C00. L'instruction écrit donc R0 à 0x40012C34. Cette correspondance relie directement la ligne C, l'assembleur et le code machine.

Effets observés sur fsm.c

À -O2, fsm_etat lit toujours les quatre variables volatile puis copie leurs valeurs dans l'état exposé ; le compilateur peut regrouper des écritures non volatile, par exemple avec STRD, mais ne supprime pas les lectures requises. Il préserve R4 dans une fonction feuille. L'alignement AAPCS de huit octets s'applique aux interfaces ; une fonction feuille peut ajuster temporairement sa pile tout en rétablissant SP au retour.

L'optimisation ne réduit pas systématiquement la taille : fsm_traiter passe ici de 364 à 372 octets. Elle peut aussi rendre la correspondance ligne/instruction moins directe et éliminer des variables visibles au débogueur. -O0 est retenu pour les captures de paramètres ; -O2 est fourni pour la comparaison et la version optimisée. Les performances réelles devront être évaluées avec DWT.

Firmware complet	text	data	bss
-O0	9852	32	3216
-O2	8516	32	3216

Les colonnes sont celles de arm-none-eabi-size. data s'ajoute au texte pour l'image de chargement ; data et bss occupent la RAM, avec les réservations du script de liens. Les pilotes assembleur inchangés, le démarrage et les bibliothèques liées font partie de ces totaux.

```
Reproduction : bash build.sh O0 puis bash build.sh O2 ; tests/run_target.sh pour le banc Cortex-M33 ; tests/qemu/debugger.gdb pour les points d'arrêt. Les exécutables ELF, journaux et fichiers de désassemblage sont inclus.
```

9. Tableau de vérification T1 à T10

Le verdict indique exactement ce qui a été observé. Un essai logiciel ne remplace pas un essai matériel. Les critères physiques restent ouverts jusqu'à l'utilisation de la plateforme.

Essai	Procédure / résultat observé	Preuve et verdict
T1 Initialisation	Construction complète et ordre d'initialisation relu. DEL, moteur et délai < 100 ms non mesurés.	cubeide_debug.log, P1. Compilation réussie ; carte à vérifier.
T2 C → ASM	GDB : R0 = 1 870 puis 39 322 360 ; PWM 75 → 150 ; SP & 7 = 0 ; enc_count lu en mémoire.	gdb_cortex_m33.log, P3/P4. Réussi en émulation.
T3 Virgule fixe	Exécution des cinq fronts imposés ; égalité aux produits Q16.16 attendus.	qemu_O0/O2.log, P5. 5/5 réussis en émulation.
T4 ADC/DMA	Moyennes 16/64 et curseur testés en mémoire ; débit calculé 3 062,79 Hz/voie.	tests_linux.log. Objet réussi ; ADC réel non mesuré.
T5 Consigne	Bornes, hystérésis, rampe jusqu'à 100 %, arrêt sous PWM et deux sens testés avec sorties substituées.	tests_linux.log, P2. Réussi sur hôte ; moteur à vérifier.
T6 Température	Calcul et indépendance des instances testés ; calibration de référence initiale conservée.	acq_v1.log, tests_linux.log. Thermomètre, doigts et pont J5 non testés.
T7 File	12 cas, dont 40 push : 32 acceptés / 8 refusés ; 10 000 cycles push/pop.	Linux et QEMU : 12/12. Windows et STM32 physique non exécutés.
T8 E-Stop	Interruption injectée pendant une écriture PWM ; maintien de l'urgence jusqu'à relâchement et Touch.	tests_linux.log. Logique réussie ; latence < 1 ms non mesurée.
T9 Télémétrie	150 trames synthétiques issues du formateur C, CRLF et cadence de dates 5 Hz ; rejet entier testé.	telemetrie_synthetique.csv, P6. Format réussi ; port série non mesuré.
T10 Comparaison	Huit cas ASM/C, effets mémoire, tailles -O0/-O2 et encodages relevés.	P4/P5/P7, listes objdump. Logiciel réussi ; cycles DWT à mesurer.

À relever sur carte : premier état et délai au reset ; valeurs ADC bas/milieu/haut ; tours DMA pendant une seconde ; température ambiante et après 30 s ; captures E-Stop pendant rampe et émission ; 30 s de CSV réel ; cycles à -O0/-O2. Le potentiomètre doit être en position basse avant les essais et le câblage J5 modifié hors tension, conformément à l'énoncé.

10. Bilan, traçabilité et références

L’implémentation logicielle et le projet STM32CubeIDE sont réalisés. Les essais reproductibles confirment le fonctionnement de la file, des objets d’acquisition, du contrôle simulé, du formateur de télémétrie et des calculs assembleur. Ils établissent également les interfaces AAPCS et la comparaison de code aux deux optimisations. La validation physique du laboratoire et la démonstration ne sont pas certifiées par ces résultats.

Journal, versions et travail d’équipe

Les checkpoints locaux acq-v1 et acq-v2 ont été créés pendant cette préparation, le 10 novembre 2026, avec l’auteur explicite « Codex (OpenAI) ». La V1 est une baseline préparée pour comparer l’évolution ; ces commits ne sont pas présentés comme l’historique de séances antérieures. Le dépôt d’origine de l’équipe est https://github.com/erudaperu/CEG3536_Lab ; aucune publication distante du Lab 3 n’a été effectuée. Un bundle Git local permet de consulter les versions.

Le module evq et les résultats Linux sont prêts pour J2. L’exécution Windows, les résultats sur la carte, les rôles effectivement tenus par chaque membre et la présence à la démonstration ne sont pas documentés ici. JOURNAL.md indique ces limites sans leur attribuer un résultat inventé.

Contenu de l’archive

Projet importable STM32CubeIDE ; firmware ELF/BIN -O0/-O2 ; sources CMSIS et pilotes conservés ; tests Linux et QEMU ; baseline acquisition V1 et annexe impérative ; journal ; fiches ADC/DMA et LPUART1 ; captures PNG ; journaux bruts et désassemblages ; ce rapport et les annexes de preuves. Les fiches antérieures retrouvées sont jointes comme références dans leur état d’origine.

Sources utilisées

[1] Énoncé CEG3536, Lab 3, Automne 2026, v1.3, sections 2 à 4 et 6 à 7 : exigences, paramètres et livrables.
[2] CEG3536_Lab3_Complement.zip : interfaces, CMSIS, banc et programme de télémétrie de départ ; adaptations décrites dans ce rapport.
[3] Projet Lab 2 local, dossier Documents/Codex/2026-10-20/g/work/CEG3536_Lab : pilotes, en-têtes, démarrage, script de liens et documents antérieurs.
[4] CEG3536_Lab2_Complement.zip : contrat original de motor_set_pwm, confirmé dans le projet retrouvé.
[5] En-têtes CMSIS/ST fournis, notamment stm32l552xx.h et core_cm33.h : types, adresses, masques et accès DWT.
[6] Sorties locales de GCC, GNU Tools for STM32, STM32CubeIDE, GDB et QEMU ; versions détaillées dans preuves/outils.txt.

Déclaration d’usage d’IA générative

Codex (OpenAI, famille GPT-6), session du 10 novembre 2026, a servi à retrouver le projet antérieur, compléter et relire les modules, préparer les essais, compiler, exécuter les bancs, collecter les traces et rédiger ce rapport. Les captures proviennent des sorties réellement enregistrées, présentées dans des vues HTML. L’assistance n’a produit ni mesure matérielle non exécutée ni attestation de participation. Chaque membre doit vérifier et pouvoir expliquer le code avant la démonstration, durant laquelle l’IA n’est pas utilisée.

Pièces complémentaires : Annexes_Preuves_CEG3536_Lab3.pdf (P1 à P7) ; Fiches_Peripheriques_CEG3536_Lab3.pdf (deux fiches). Les fiches sont présentées dans un format autonome ; le modèle Brightspace n’était pas présent dans les fichiers retrouvés.